

En esta práctica se pide la implementación de un conjunto de funciones y un programa principal para el manejo de cadenas de caracteres junto con matrices. Las cadenas van a ser matrículas de tráfico con el formato E-DDDD-LLL, donde D es un dígito y L una letra del alfabeto (sin incluir la Ñ), siendo válidas todas las letras aunque en la realidad haya algunas letras que no se usen. Por ejemplo, E-2166-BRB o E-8882-FYW serían matrículas válidas. Las funciones que se pide implementar son:

- **rellenar** una matriz de 40 matrículas válidas usando la función aleatorio de las prácticas anteriores.
- **imprimir** el contenido de la matriz.
- **comparar** dos matrículas. Esta función devolverá un **valor negativo** si la primera matrícula es más antigua que la segunda, un **valor positivo** si la primera matrícula es más reciente que la segunda y **0** si son iguales.
- **mas_antigua**, que devuelve la matrícula más antigua de las almacenadas en la matriz.

El programa principal deberá rellenar la matriz con las 40 matrículas, imprimir la matriz y también la matrícula más antigua.

Una posible ejecución de la misma es la siguiente:

```
E-6393-HLQ
E-0041-IUS
E-5652-QST
E-6809-NZG
E-5843-RLI
E-2997-GKC
E-4271-UXB
E-2112-GPE
E-1350-QCL
E-7501-FFK
E-8891-ZZL
E-5562-MTE
E-4893-LFB
E-3059-WLG
E-1308-ERL
E-4818-WMD
E-0142-QNA
E-3549-HUL
E-6636-IFU
E-8676-MES
E-5212-TDE
E-6634-TQT
E-2757-EUS
E-7888-TIM
E-7312-UXC
E-3253-QKH
E-1306-GGA
E-6371-TBX
E-9286-NNC
E-7857-IEW
E-7872-RFU
E-6286-CXJ
E-9915-YOH
E-2899-XJI
E-4422-WET
E-2222-DFB
E-3034-RCW
E-9294-OKM
E-7057-RAM
E-6895-OPA

La matricula mas antigua es E-6286-CXJ.
```

SOLUCIÓN:

```
// includes y defines
```

```
#include <string.h>
#include <time.h>
#include <math.h>
#define F 40
#define C 11//por el \0
```

```
// prototipos
```

```
void rellenar(char matriz [F][C]);
void imprimir(char matriz[F][C]);
int comparar(char *cadena1[C], char *cadena2[C]);
char* mas_antigua(char matriz[F][C]);
```

```
// main
```

```
int main(){
    char mat[F][C];
    rellenar(mat);
    imprimir(mat);
    fflush(stdin);
    printf("\nLa matricula mas antigua es: %s",mas_antigua(mat));
    srand(time(NULL));
    return 0;
}
```

```
// definición de funciones
```

```
int aleatorio(int inf, int sup){
    int aux = inf;
    if (inf > sup)
        { inf = sup;
          sup = aux;
        }
    return (rand()%(sup-inf+1)+inf);
}
```

```
void rellenar(char matriz[F][C]){
    for(int i=0;i<F;i++){
        matriz [i][0] = 'E';
        matriz [i][1] = '-';
        matriz [i][6] = '-';
        matriz [i][10] = '\0';
        for(int j=0;j<C;j++){
            if(j>1 && j<6){
                matriz[i][j] = aleatorio('0','9');
            }else if(j>6 && j<10){
                matriz[i][j] = aleatorio('A','Z');
            }
        }
    }
}
```

```

    }
}
}
void imprimir(char matriz[F][C]){
    for(int i = 0;i<F;i++){
        printf("%s\n",matriz[i]);
    }
}

int comparar(char *cadena1[C], char *cadena2[C]){//recuerdad
    int aux = strncmp(cadena1+7,cadena2+7,3);//pongo los numeros porque si puesiera cadena1[i]
    estaría recaudando la información y de la manera que lo pongo solo estoy apuntando a la información
    para comparar
    if (aux == 0)
        aux = strncmp(cadena1+2,cadena2+2,4);
    return aux;
}

char* mas_antigua(char matriz[F][C]){//lo que hace es guardar el valor del return en una dirección
de memoria, ya que de esta forma no se borra el valor que es lo que pasa en una función normal y
puedes itilzarlo posteriormente.
    int i,pos_antigua = 0;
    char masAnt[C];
    strcpy(masAnt,matriz[0]);
    for(i=1;i<F-1;i++)
        if(comparar(matriz[i],matriz[pos_antigua])<0)
            pos_antigua = i;
    return matriz[pos_antigua];
}

```